VeriQ

Your complete project guide, in easy language

Tata Elxsi TELIPORT Season 4 · Case Study 3 · Adaptive QoS Engine for Mixed Home Broadband Traffic

What this guide is. Everything we researched and designed for your project, explained step by step. You do not need any networking background to start. Every hard word is explained the first time it appears, and there is a glossary at the end.

How to read it. Read Chapters 1–4 first: they give you the full picture in about 20 minutes. Chapters 5–6 teach the networking basics. Chapters 7–12 explain VeriQ part by part. Chapters 13–17 tell you exactly what to build, how to test it, and how to present it.

> **Honesty note (please read)**
>
> VeriQ is currently a **design**. The only real measurements so far come from the Day-0 lab script, which was run in a test sandbox (Chapter 13). All other numbers in this guide are **targets**: goals you must prove with your own experiments. Judges respect teams who clearly separate the two.
>
> Prepared on 11 October 2026. Company facts come from public sources listed in Chapter 19.

Contents

1. The whole project on one page 3
2. The competition: TELIPORT Season 4 4
3. Who is Tata Elxsi, and why do they care? 5
4. Why we chose Case 3 (the scorecard) 6
5. Networking basics in easy language 7
6. The tools: Linux traffic control explained 9
7. What Case 3 actually asks for 10
8. VeriQ: the big idea 12
9. VeriQ part by part (modules M1–M9) 14
10. Finding the cause before acting (fault localisation) 16
11. Classification, confidence and saying “I'm not sure” 17
12. Fairness, guarantees and honest statistics 18
13. Measuring user experience 19
14. The Day-0 lab you already have 20
15. What to build, and when 22
16. Making it unforgettable (and credible) 24
17. Your slides 25
18. Questions judges will ask, and how to answer 27
19. Honest guarantees, limits, glossary and sources 28
20. A rival team's prototype, and what we learned 31

Appendix: the other cases, and what the external review changed 33

1. The whole project on one page

You are taking part in **TELIPORT Season 4**, a national case competition run by **Tata Elxsi** for engineering students. You chose **Case Study 3: an Adaptive QoS Engine for home broadband**.

The problem in one sentence

When someone in a house starts a big download or a cloud backup, video calls and games become laggy, even though the internet plan is fast. The reason is **waiting in queues**, not lack of speed.

Our solution in one sentence

VeriQ is software that runs on the home router. It watches traffic without reading anyone's private data, figures out what kind of traffic each flow is, tracks how fast the internet line really is right now, keeps the queues short, protects important things like a work call when the user asks, and **explains every decision with evidence**.

> **The pitch line**
>
> “VeriQ adapts, measures, and explains: every decision comes with evidence, and every claim with a method.”

Why this can win

- **Few teams will pick it**: most students have never worked with Linux networking. We estimate only 5–10% of teams choose Case 3.
- **Tata Elxsi cares about it**: its Media & Communications business (home routers, broadband, network software) is its fastest-growing unit (+22% year on year in Q1 FY27).
- **It is judged on numbers** (delay, fairness, speed of reaction), so a careful team wins clearly.
- **It can be demonstrated live**: a judge presses a button, the internet slows down, and the video call keeps working.

What you must do next (in order)

1. Confirm the registration and Round 1 deadlines on the official portal (sources say 23–24 Oct 2026).
2. Run the Day-0 lab script on an Ubuntu laptop and get **your own** numbers (Chapter 13).
3. Put your team name, college and your own numbers on the Round 1 slide, and submit it as a one-page PDF.
4. Start building the MVP right after Round 1. Don't wait for the shortlist (Chapter 14).

2. The competition: TELIPORT Season 4

Item	Details	What it means for you
Who can enter	Pre-final and final-year B.Tech / M.Tech / dual degree; teams of 2–3 from the same college	Mixing branches is allowed (e.g. CSE + ECE)
Round 1	Pick ONE case. Submit a **single slide as a PDF**	The whole shortlist depends on one slide. It must be sharp and honest.
Round 2	10-slide deck + video/prototype. Window: 14 Nov – 13 Dec 2026	About 4 weeks of building
Round 3	Grand Finale: present to Tata Elxsi senior leaders	Live demo + tough questions
Prizes	Winner ₹2 lakh · 1st runner-up ₹1.5 lakh · 2nd runner-up ₹1 lakh	
Jobs	National finalists get **pre-placement offers**; Round 2 teams get pre-placement interviews	Even reaching Round 2 is valuable
Ownership	Submitted ideas become Tata Elxsi's property	Keep your code repository private
Size	Season 2 had 6,000+ teams	Standing out is everything

Deadlines conflict

One listing says registration closed on 10 Oct; others say 23 Oct, with Round 1 due 24 Oct 2026. Check the official portal today.

3. Who is Tata Elxsi, and why do they care?

Tata Elxsi is a Tata group company that designs technology for other companies: car software, TV and broadband software, and medical-device engineering. In FY26 it earned about ₹3,757 crore. In Q1 FY27 (April–June 2026) it crossed ₹1,000 crore in one quarter for the first time.

Business unit	Share of revenue	How it is doing	Connection to you
Transportation (cars, aerospace)	~55%	+13% YoY	Case 5 (VehicleGPT)
Media & Communications (MCV)	**~33%**	**+22% YoY, fastest growing**	**Case 3: your case**
Healthcare & Life Sciences	~9%	Fell during FY26, now pivoting to AI	Case 4 (labeling)

What MCV actually builds (useful to mention)

- **RDK-B**: open-source software that runs inside home broadband routers. Tata Elxsi builds and ports it, e.g. the first RDK-B on Qualcomm's 5G fixed-wireless platform, where QoS was one of the operator features.
- **NEURON**: Tata Elxsi's platform for 'autonomous networks' that is **intent-based** and **closed-loop**. VeriQ follows the same thinking inside a single home. That is a good talking point, but don't claim VeriQ plugs into NEURON.
- Broadband customers include large operators in Europe (e.g. Sky is a NEURON customer).

> **What the judges want to see**
>
> Engineering that a broadband operator could trust: measured results, safe behaviour when things go wrong, and an honest description of limits. Fancy words without evidence will hurt you.

4. Why we chose Case 3 (the scorecard)

We rated all five cases from 1 to 10 on several things, then combined the scores with weights that match what you asked for: standing out, low competition, relevance to Tata Elxsi and wow factor.

Case	Topic	Weighted score	Estimated share of teams	Short verdict
CS4	Medical device labeling (AI)	8.40	15–20%	Strong, was our first pick
CS3	**Adaptive home QoS**	**8.15**	**5–10%**	**Your pick: least crowded**
CS5	VehicleGPT (LLM in a car)	6.65	25–30%	Very crowded (AI hype)
CS2	Battery design platform	5.65	15–20%	Crowded with EEE/mech teams
CS1	Predictive maintenance	4.75	25–30%	Thousands of tutorials exist

Case 3 and Case 4 were very close. Case 3 has the fewest competitors, belongs to the fastest-growing business, and is judged on hard numbers. Its risk is that it is technically hard, which is why this guide teaches the basics carefully.

> **Remember**
>
> The 'share of teams' numbers are our estimates. No public data exists.

5. Networking basics in easy language

5.1 Packets

Everything on the internet (a video, a voice call, a file) is cut into small pieces called **packets**, usually up to about 1,500 bytes each. Each packet travels separately and is put back together at the other end.

5.2 Bandwidth vs latency: the most important idea

Word	Simple meaning	Example
Bandwidth (Mbps)	How MUCH data can flow per second: the width of the pipe	A 100 Mbps plan
Latency / delay (ms)	How LONG one packet takes to arrive	20 ms to a server in Mumbai
RTT (round-trip time)	Time to go there and come back (what 'ping' measures)	40 ms
Jitter	How much the delay keeps changing	Delay jumping 20 → 90 → 30 ms
Packet loss	Packets that never arrive	2% loss makes a call robotic

A video call does not need much bandwidth (about 2–3 Mbps), but it needs **low and steady delay**. A download needs lots of bandwidth but doesn't care about delay. Problems start when both share one line.

5.3 The home network



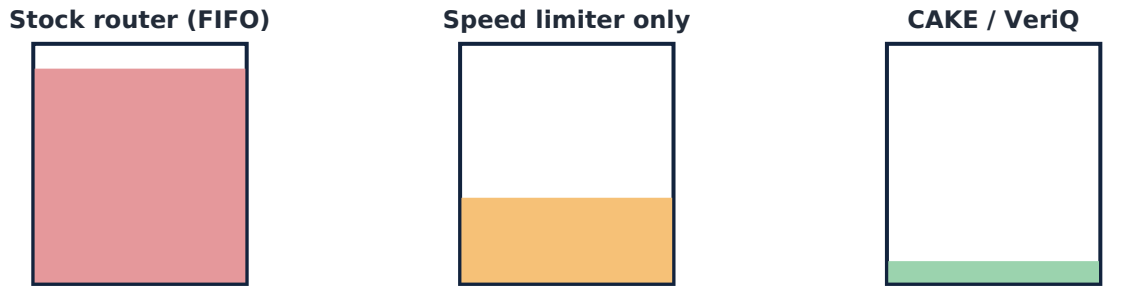
Fast inside the house (LAN) · narrowest pipe = the internet line (WAN)

Traffic waits in the queue in front of the narrowest pipe.

Inside the house, Wi-Fi and cables are fast. The slowest part is usually the internet line itself (the **WAN**, Wide Area Network). The modem and the ISP equipment in front of that line have a **buffer**: a waiting area where packets queue when they arrive faster than the line can send them.

5.4 Bufferbloat: the real enemy

Think of a toll plaza. If 100 cars arrive per minute but only 60 can pass, a queue builds. A big download (using a protocol called TCP) is designed to keep sending faster and faster until the line is full, so it **always** fills the queue. If the waiting area is huge, every packet of your video call has to wait behind thousands of download packets. That extra waiting is called **bufferbloat**.



> **Real number from our lab**
>
> With an emulated 5 Mbps upload and a 300 ms modem buffer, a cloud-backup-style upload added **266 ms** of delay (95th percentile) to everything else. A good call needs less than about 30–50 ms of extra delay.

5.5 Why 'just prioritise the call' does not work by itself

If the queue lives in the ISP's modem, your router cannot reorder it: the router is not the slowest point, so it never holds a queue. The trick used by experts is to make the router send **slightly slower** than the internet line, e.g. 95% of its speed. Now the queue forms inside the router, where smart software can keep it short and fair. This is called **shaping**, and it is the key idea behind this whole project.

> **But there is a catch**
>
> To shape at 95% you must know the line's real speed. On cable, 5G home internet and Wi-Fi the speed changes during the day. Too high a shaper and the bloat comes back; too low and you waste bandwidth. **Tracking the real speed is the heart of the problem.**

5.6 Other words you will meet

- **Upload vs download** (upstream/downstream): home upload is usually much slower, so uploads (backups, posting videos) cause the worst bufferbloat.
- **Flow**: one conversation between two programs, e.g. one video call or one download.
- **Encryption, TLS and QUIC**: almost all traffic is encrypted today, so you cannot (and must not) read its content. You can still see packet sizes and timing.
- **Port**: a number that once told you the app (e.g. 80 = web). Today almost everything uses port 443, so ports tell you little.
- **DSCP**: a small 'priority label' field in each packet. Most apps don't set it.
- **IPv4 / IPv6**: the two versions of internet addresses. The case asks you to support both.
- **TCP / UDP**: TCP is reliable and slows down when packets are dropped (downloads, web). UDP is used by calls and games.

6. The tools: Linux traffic control explained

Home routers usually run Linux. Linux has a built-in system called **tc** (traffic control) that decides how packets wait and leave. Each waiting-room rule is called a **qdisc** (queueing discipline).

qdisc	In simple words	Good at	Weak at
FIFO	First in, first out. One line, no intelligence	Simple	Bufferbloat
fq_codel	Gives each flow its own small lane and drops packets that wait too long	Short queues, fairness between flows	Useless if the queue is in the modem
CAKE	fq_codel plus a built-in speed limiter (shaper), per-device fairness and 4 priority classes ('tins')	The best all-in-one for homes	Needs the right speed setting; tins are 'preference', not guaranteed minimums
HTB	A tree of classes, each with a guaranteed rate and a maximum rate	Real guaranteed minimum bandwidth	More complex to configure
tbf	Simple speed limiter with a fixed-size queue	Emulating an ISP line in our lab	—
netem	Adds fake delay or loss	Emulating distance to internet servers	—

Other important tools

- **Network namespaces**: lets one Linux laptop behave like many separate computers with their own network. We use 4 of them to build a whole fake home network (Chapter 13).
- **eBPF**: small safe programs that run inside the Linux kernel at every packet. Very fast. A stretch goal for collecting packet features.
- **netlink / pyroute2**: the way a Python program talks to the kernel to change qdiscs.
- **iperf3**: creates heavy traffic (fake downloads). **ping** measures delay.
- **cake-autorate**: an existing open-source tool that changes CAKE's speed setting automatically. It is our strongest competitor and a **baseline** we must compare against.
- **OpenWrt SQM**: the popular router setup that applies CAKE at a fixed speed. Another baseline.

> **Key insight for interviews**
>
> CAKE already solves a lot. Judges will ask: **“What did you add on top of CAKE?”** Our answers: fault diagnosis before acting, guaranteed-rate intents, payload-free classification, evidence receipts, and an honest comparison with the real cake-autorate.

7. What Case 3 actually asks for

Here is the case statement translated into plain language.

The case says	In simple words
Intent-aware QoS controller	A program that controls traffic and can follow user wishes like 'prioritise my work call'
Classify traffic without reading private payloads	Guess whether a flow is a call, game, video or download using only sizes and timing, never content
Estimate link capacity	Keep track of how fast the internet line really is right now
Apply queueing and shaping	Use qdiscs like CAKE/HTB to keep queues short and fair
Verify whether the policy improved user experience	Measure whether calls and videos actually got better
Temporary intent while retaining fairness	Priority for a limited time, without starving others
Compare AI with a deterministic baseline	Show your smart method beats simple fixed rules, with numbers

The three example situations

1. A big download starts during a video conference: the call must stay smooth.
2. The internet speed drops from 100 to 20 Mbps: the controller must adjust quickly instead of building a huge queue.
3. Three TVs stream while someone plays an online game: everyone must get fair service and the game must stay responsive.

The rules (constraints)

- Don't decrypt traffic. Use Linux tc or another open mechanism. Support IPv4 and IPv6 where available.
- Be able to roll back a policy; low-priority traffic must never starve.
- The classifier must show its confidence and allow corrections.
- Document licences; include data and settings so others can reproduce your results; label emulation as emulation; no real secrets in the code.
- Automatic changes must be limited, visible and reversible, and failure must lead to a known safe state.

What they want delivered

- Traffic classifier + capacity estimator
- Policy engine + enforcement
- Dashboard: latency, jitter, loss, throughput, queue depth, fairness
- Automated baseline-vs-optimised experiments
- API or UI for temporary intents
- Architecture diagram

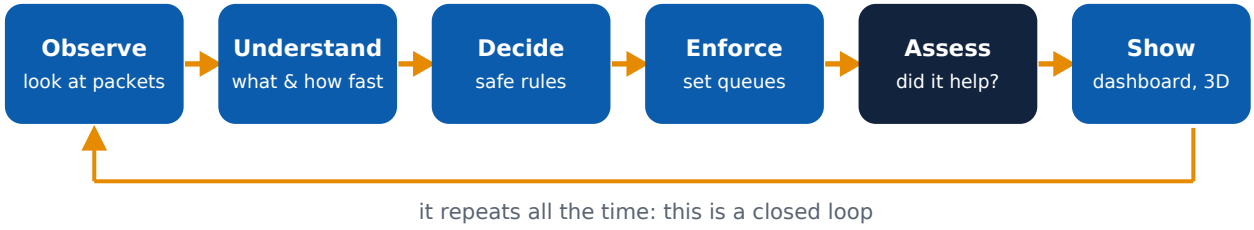
- Setup guide with exact versions
- Scripted demo: reset → cause a problem → collect evidence → generate report
- Known-limitations section

How they will judge

- Accuracy of classification and capacity estimation
- Reduction in call/game latency, jitter, loss
- Fairness and no starvation
- Speed of adaptation and stability
- CPU cost, reproducibility, clarity

8. VeriQ: the big idea

VeriQ is a controller that runs on the home router and repeats one loop all the time:



it repeats all the time: this is a closed loop

The three speeds (the most important design idea)

1. Fast guardrails	**every 0.1–1 s** keep delay low, react to speed drops
2. Outcome receipts	**every 15–60 s** improved / degraded / inconclusive
3. Controlled experiments	**in the lab, many runs** the ONLY source of proof

1. **Fast guardrails** (every fraction of a second): simple, safe rules that keep delay low and react to speed drops immediately. Safety never waits for statistics.
2. **Outcome receipts** (every few tens of seconds): after each change VeriQ writes a short report: did calls and videos get **better**, **worse**, or is it **inconclusive**? It is like a shop receipt for every decision.
3. **Controlled experiments** (in the lab, repeated many times): the only place where we claim 'VeriQ is better', because only there can we hold everything else equal.

> **Why not just 'test each change for 15 seconds and keep it if better'?**
>
> Imagine the internet speed falls from 100 to 20 Mbps. VeriQ correctly lowers the shaper. Calls are still worse than before, because the line itself got worse. A naive before/after test would wrongly say 'this change made things worse' and undo a good decision.
>
> So VeriQ says **'inconclusive'** whenever conditions changed during the check, and any undo is recalculated for the **current** speed. This correction came from an external review and makes the design much stronger.

What makes VeriQ different from other teams

Most teams will do	VeriQ does
Priority by port numbers	Learns from packet sizes and timing, says 'not sure' when unsure
Fixed speed limit	Tracks the real speed with a confidence range, and finds the **cause** first
'AI decides priority' with no proof	Every decision has a receipt; claims come from repeated experiments

Most teams will do	VeriQ does
Ping graphs as proof	Counts real video-call freezes and video stalls
Strict priority (starves others)	Guaranteed minimums with a budget; progress is measured for everyone
Static dashboard	3D home model + replay that separates measured, estimated and illustrative

9. VeriQ part by part (modules M1–M9)

Each module is a separate piece you can build and test on its own. The case explicitly asks for this.

M1 · Flow Observer: 'the eyes'

What it does	Watches packets going through the router and records, for each flow: who is talking, packet sizes, directions and the time gaps between packets, for the first few packets. It never looks at content.
How	Start with simple capture in Python (AF_PACKET). Move to eBPF later only if CPU use requires it.
Example	Phone starts a Meet call → M1 sees many small UDP packets every ~20 ms in both directions.

M2 · Classifier: 'the brain that names traffic'

What it does	Guesses the type of each flow: REALTIME (calls), GAMING, STREAMING (Netflix-like), INTERACTIVE (web), BULK_DOWN, BULK_UP, or UNKNOWN.
How	A small machine-learning model (gradient-boosted trees) that decides after 5, 10, 20 or 30 packets. When it is not sure, it answers UNKNOWN and the flow gets normal treatment, never priority.
Example	Steady small packets at a regular rhythm → probably REALTIME. Big packets one way for minutes → BULK.

M3 · Capacity Estimator + Fault Localiser: 'the doctor'

What it does	Estimates the current internet speed (with a ± range) and diagnoses **where** a problem is before VeriQ acts. Chapter 10 explains this in detail.
How	Sends tiny timing probes to a nearby server and to far servers, watches its own queue, and uses change detection to notice sudden speed drops. On an idle line (no traffic to learn from) it sends a short burst of paced test packets to a reflector server (the SLoPS method by Jain & Dovrolis) to find the speed.
Example	Speed falls 100 → 20 Mbps: near and far probes both slow down while the router's own queue is empty → diagnosis 'WAN capacity dropped'.

M4 · Policy Engine: 'the decision maker'

What it does	Chooses settings: shaper speed, which class goes where, active intents.
How	Shaper speed = estimated speed minus a safety margin that grows when the estimate is uncertain (written $\mu - k\sigma$: μ = best guess, σ = uncertainty). It cuts quickly on drops and increases slowly.
Example	Estimate 20 ± 1 Mbps → shaper ≈ 18.5 Mbps.

M5 · Enforcer: 'the hands'

What it does	Applies settings to Linux. Two modes: **CAKE** by default (short fair queues, soft priority) and **HTB guaranteed mode** when someone has an active intent (real minimum speeds).
How	Changes are made step by step in a safe order, saved with a version number, read back to confirm, and repaired if they don't match. A watchdog switches to a conservative SAFE setting if VeriQ crashes.
Example	User protects the laptop → HTB gives the laptop class a guaranteed 4 Mbps.

M6 · Outcome Assessor: 'the judge'

What it does	Writes the experience receipt after each change: improved / degraded / inconclusive, with the reason.
How	Compares real metrics (call freezes, video stalls, delay, progress of every class) in comparable time windows only. Chapter 12 explains the statistics.
Example	'WAN drop detected · shaper 92 → 18.6 Mbps · 0 freezes · outcome: IMPROVED vs earlier minutes at the same speed.'

M7 · Intent Leases: 'the user's wishes'

What it does	Lets a user say 'protect my work call for 45 minutes'. This becomes a **lease**: it is authenticated, it always expires, and it is paid from a **budget** so others keep enough.
How	Admission control: if the request would push total guaranteed speed above the budget, it is partly granted or refused, with a clear reason.
Example	Two people ask for priority at the same time → both get part of the budget, and the downloads still move.

M8 · Dashboards + 3D twin: 'the window'

What it does	Grafana charts for engineers, plus a 3D model of the house for everyone else.
How	Every element is marked **measured** (solid), **estimated** (outlined with a confidence band) or **illustrative** (animation). The ISP queue is always 'inferred'. In the lab it is shown next to the true value, so judges can see how accurate VeriQ is.
Example	Replay: drag the timeline to the moment the speed dropped and watch every panel change together.

M9 · Evidence Harness: 'the science lab'

What it does	Runs scenarios automatically, compares VeriQ with the real competitors, and creates a PDF report.
How	Fake home network in namespaces; real video calls (WebRTC) and real video players (DASH) in a headless browser; at least 10 runs per setup in random order; confidence intervals.
Example	'make scenario S2' → a report: VeriQ vs FIFO vs fq_codel vs SQM-CAKE vs cake-autorate.

10. Finding the cause before acting (fault localisation)

A doctor diagnoses before giving medicine. VeriQ does the same. Slow internet can have very different causes, and the right action is different for each one.

How it tells them apart: VeriQ looks at (1) its own queue, (2) timing probes to a **near** point (the ISP's first router), (3) timing probes to several **far** servers, and (4) Wi-Fi statistics where available.

What VeriQ sees	Diagnosis	What VeriQ does
Its own queue grows, outside delay is normal	The router is the bottleneck (this is the healthy, intended case)	Nothing special: CAKE handles it
Its own queue is empty, near AND far delay both rise	The internet line got slower	Cut the shaper fast, then probe upward slowly
Only far servers are slow, near point is fine	Problem somewhere far away on the internet	**Hold**: cutting would waste speed and not help
Internet fine, but one Wi-Fi device has many retries	Wi-Fi problem	Don't touch the WAN shaper; show a warning
One probe server disagrees with all the others	A misleading measurement	Trust that server less; do nothing
Signals contradict each other	Unknown	Hold the current settings and raise an alert

> **Why judges will like this**
>
> Many controllers cut speed whenever delay rises, even when the problem is far away, and waste bandwidth. VeriQ checks first, and you can prove it: in the lab you know the true fault, so you can score how often VeriQ's diagnosis was right.

11. Classification, confidence and saying “I'm not sure”

How you can classify without reading content

Different apps have different 'body language'. Calls send small packets at a steady rhythm in both directions. Games send tiny packets very often. Video streaming downloads big chunks every few seconds, then pauses. Downloads send big packets continuously. These patterns survive encryption.

Early classification

You want to know quickly. VeriQ tests decisions after 5, 10, 20 and 30 packets and picks the earliest point that is reliable enough on validation data.

Conformal prediction, explained simply

Normally a model gives one answer, e.g. 'REALTIME'. **Conformal prediction** instead gives a **set** of possible answers that contains the true one about 95% of the time, e.g. {REALTIME} or {REALTIME, GAMING}. If the set has exactly one answer, VeriQ acts on it. If it has several, VeriQ says 'not sure' and gives normal treatment.

> **Report these numbers separately (do not mix them up)**
>
> 1. **Coverage**: how often the set contains the true answer (target about 95%).
> 2. **Selective accuracy**: how often the single answers we act on are correct.
> 3. **Abstention rate**: how often VeriQ says 'not sure'.
> 4. All three again on **different** traffic (another day, other apps). This is called testing under **shift**.
>
> Mistake to avoid: saying '95% coverage' means '95% of our decisions are correct'. It does not.

Data

- **Main data**: traffic you generate yourself in the lab, so you know the true label of every flow.
- **Extra benchmark**: CESNET-QUIC22, a public dataset of 153 million real QUIC flows. Its labels are service names, so you must write down how you map them to your categories.
- **Baseline**: simple port rules. You must show your model beats them, both in accuracy and in call quality.

12. Fairness, guarantees and honest statistics

Soft priority vs real guarantees

CAKE's four 'tins' give **preference**: important traffic goes first, but only up to a share. That is not a guaranteed minimum. When a user has an active intent, VeriQ switches to **HTB**, where each class gets a real guaranteed rate and can borrow unused speed up to a maximum.

The promotion budget

Total guaranteed speed for all intents can never exceed, say, 60% of the shaper speed. The rest stays available for everyone else. A request that does not fit is partly granted or refused, with a reason.

Starvation is measured, not assumed

A setting that says 'minimum 1 Mbps' does not prove a download keeps moving. VeriQ measures each class's actual **progress** (bytes delivered per time window) and reports it.

Statistics in plain words

Term	Simple meaning	Why we use it
Median / p95	Middle value / value that 95% of samples are below	p95 shows the bad moments people notice
Confidence interval (CI)	A range that probably contains the true value	Shows how sure we are
Bootstrap	Re-sample your data many times to estimate the CI	Works without strong assumptions
Block bootstrap	Bootstrap that keeps nearby samples together	Delay samples next to each other are related
Randomised order	Run the setups in random order	So 'time of day' cannot bias results
≥10 runs	Repeat each experiment many times	One run can be luck
Holm correction	Stricter rules when you test many things at once	Prevents false 'wins'
Ablation	Turn off one part of VeriQ and measure again	Proves which part actually helps

> **Golden rule**
>
> Receipts help operate the system. **Proof** comes only from repeated, randomised lab experiments. Say exactly this if a judge asks.

13. Measuring user experience

Metric	What it is	Type
WebRTC freezeCount, freeze duration	How often and how long the video in a real call froze	Measured
Concealed samples	Audio the call had to 'invent' because packets were late or lost	Measured
DASH rebuffer events	How often a streaming video stopped to load	Measured
Queuing delay p50/p95/p99, loss	Waiting time and lost packets	Measured
Progress per class / host	Is everyone still moving?	Measured
Jain fairness index	A number from 0 to 1; 1 = perfectly fair share	Measured
MOS via ITU-T E-model	Estimated call-audio quality score (1–5) from a planning model	**Estimated**
RPM (responsiveness)	Round-trips per minute under load, from an IETF draft	Scheduled test only

Two important rules

Always label MOS as **estimated**; real freezes and stalls are stronger evidence.

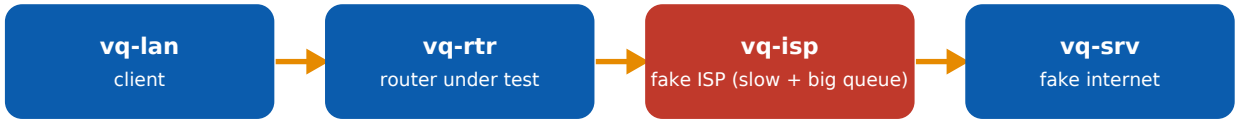
Run the RPM test on a schedule. It creates heavy load itself, so never run it secretly during someone's call.

The test scenarios (targets, not results yet)

#	Situation	Target
S1	Big download during a video call	0 freezes; extra delay p95 ≤ 10 ms; download keeps ≥ 85% of the leftover
S2	Internet drops 100 → 20 Mbps mid-call	Correct diagnosis; settles within 2 s; no swinging up and down
S3	3 TVs + a game	Game extra delay p99 ≤ 20 ms; no video stalls; fairness ≥ 0.9
S4	Cloud backup upload during a call	Same as S1, for upload
S5	'Protect laptop 45 min'	Guarantee honoured; others keep moving; lease ends on time
S6	A far-away server is slow	Diagnosed as remote; shaper is **not** cut
S7	A probe server gives wrong readings	Ignored; no action
S8	VeriQ crashes	SAFE setting engages; time-to-safe measured
S9	IPv6-only device	Same behaviour as IPv4

14. The Day-0 lab you already have

The file **lab/quickstart_bufferbloat.sh** builds a complete fake home network inside one Linux laptop and measures bufferbloat. This is your first real evidence.



4 'mini computers' (network namespaces) inside ONE Linux laptop, joined by virtual cables

What it does, step by step

1. Creates 4 network namespaces (client, router, ISP, internet) connected by virtual cables, with IPv4 and IPv6.
2. Makes the 'ISP' slow (20 Mbps down, 5 Mbps up by default) with a deliberately huge 300 ms queue, like many real modems.
3. For each router setup ('arm'), measures idle delay, then starts 4 heavy TCP flows and measures delay again.
4. Repeats for download and upload, then prints a table and saves raw data plus an 'environment fingerprint'.

The four arms

Arm	Meaning	Expected result
fifo	Normal router	Big extra delay, especially on upload
shortfifo	Speed limiter with a short dumb queue	Helps a bit; blunt; not fair
fq_codel	Smart queue but no speed limiter	Almost no help: the queue is still in the modem
cake	Speed limiter + CAKE	Extra delay drops to a few ms (your key evidence)

Real results so far (sandbox run, emulated, IPv4 only)

Arm	Download: extra delay p95	Upload: extra delay p95
fifo	63.6 ms	**266.1 ms**
shortfifo	55.4 ms	**39.6 ms**
fq_codel, cake	not run: the sandbox's kernel lacks them	→ run on your laptop

How to run it on your laptop (Ubuntu 24.04)

```
sudo apt install iproute2 iperf3 iputils-ping ethtool python3
sudo modprobe sch_cake sch_fq_codel sch_netem sch_tbf
#   if modprobe fails: sudo apt install linux-modules-extra-$(uname -r)
```

```
sudo lab/quickstart_bufferbloat.sh
sudo DOWN_MBIT=100 UP_MBIT=20 lab/quickstart_bufferbloat.sh fifo cake
```

Read the column **added_p95_ms**: it is the extra delay a call or game feels. Put your fifo-vs-cake numbers on the Round 1 slide in place of the 266 ms sandbox number.

15. What to build, and when

MVP spine: must work end-to-end by 13 December

1. Lab + scenario runner + real baselines (SQM-CAKE, cake-autorate v3.2)
2. Observer (Python capture) + small calibrated classifier that can abstain
3. Capacity estimator + fault localiser (WAN / remote / own queue / bad probe)
4. Enforcer: CAKE default + HTB guaranteed mode + leases with a budget
5. Receipts: improved / degraded / inconclusive
6. Real WebRTC call + DASH video in a headless browser
7. Grafana dashboards + recorded replay
8. Full evaluation: 5 arms + ablations, ≥10 random-order runs, report

Stretch goals, in priority order

1. 3D twin (replay first, live later)
2. Physical Raspberry Pi 5 kit with fault buttons
3. eBPF observer
4. Conformal prediction on CESNET-QUIC22
5. Wi-Fi airtime control
6. TR-181 / USP management mapping
7. L4S / DualPI2
8. Multi-gig scaling study
9. TLA+ model of leases

Timeline

Dates	Goal
11–13 Oct	Confirm deadlines; set up Ubuntu; run the Day-0 lab
14–22 Oct	Finish the Round 1 slide with your own numbers; do the reading list
≤ 24 Oct	Submit Round 1
25 Oct – 13 Nov	Build early: lab runner, observer, enforcer + watchdog, estimator v1
14–20 Nov	Classifier + abstention; fault localiser scoring
21–27 Nov	Policy engine, receipts, leases + HTB
28 Nov – 4 Dec	WebRTC/DASH harness, Grafana, 3D twin v1, physical kit
5–13 Dec	Full evaluation, report, video, 10-slide deck, rehearsal

Who does what (team of 3)

Role	Owns
Systems / datapath	Lab, observer, enforcer, watchdog, physical kit, CPU measurement
ML / control	Classifier, estimator, fault localiser, control rules, statistics

Role	Owns
Platform / experience	Leases API, Grafana, 3D twin, report generator, video, slides

16. Making it unforgettable (and credible)

#	Asset	Why it works	Cost
1	Physical kit with a red 'WAN DROP' button	The judge presses it and sees the call survive	~₹11–18k
2	Split-screen proof video	Normal router freezes, VeriQ stays smooth, same download	₹0
3	One-command lab + PDF evidence report	How industry validates networking products	₹0
4	3D twin with replay	Makes invisible queues visible	₹0
5	Chaos button (crash VeriQ live)	Shows safe failure	~₹300
6	Judge picks the fault	Proves VeriQ reacts to real signals, not a script	₹0

Physical kit shopping list (approximate)

Item	Role	~₹
Raspberry Pi 5 (4 GB) + cooler + power + microSD	VeriQ router	8,500
USB 3.0 → 2.5 GbE adapter	Second network port	2,000
5-port gigabit switch	Home network	1,200
Old laptop with 2 network ports (or a 2nd Pi)	Fake ISP + local servers (Jitsi call server, video files)	0 / 8,500
Arcade buttons + wires	WAN DROP and chaos buttons	300
Optional 7-inch HDMI screen	Shows the 3D twin	4,000

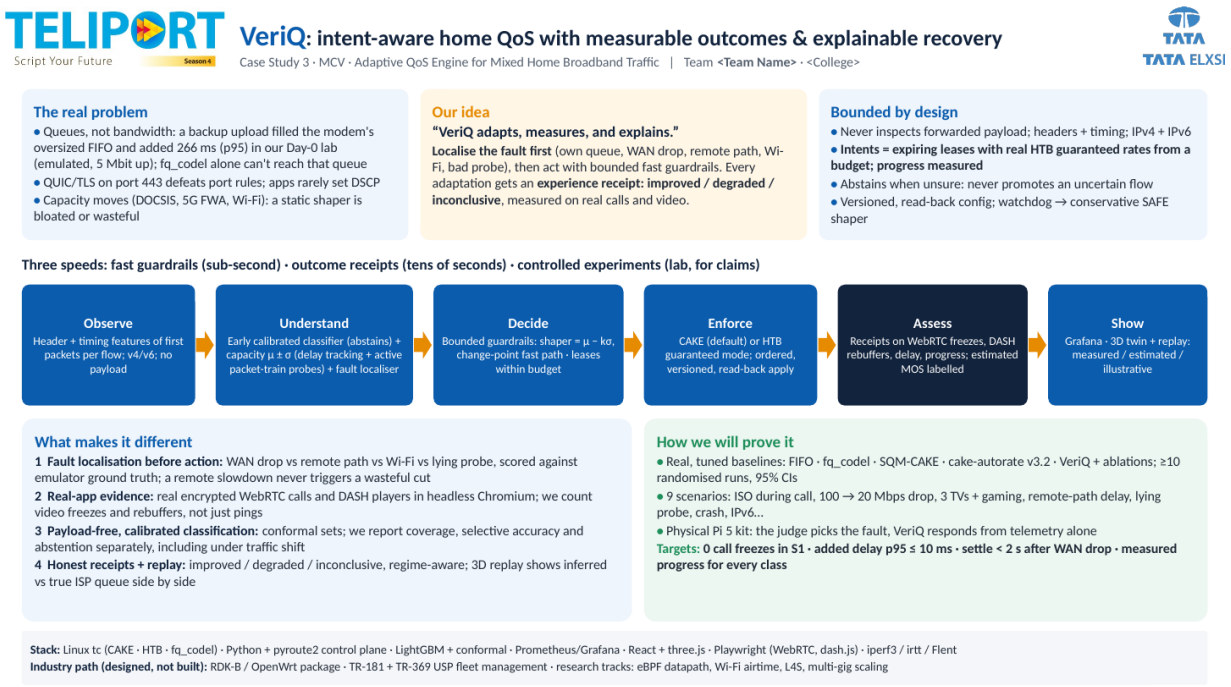
Run everything offline with a local Jitsi server, so the venue Wi-Fi cannot ruin the demo.

Credibility rules for every slide

- Tag every visual as measured, estimated or illustrative.
- Targets go on design slides; result slides show only measured numbers with run count and confidence intervals.
- Show at least one result where VeriQ did **not** beat a baseline, and explain why.
- Never claim 'no drawbacks' or '100% perfect'. Engineers distrust that.

17. Your slides

Round 1: the single slide (already built)



File: **submission/Round1_Case3_VeriQ.pptx** (slide 2 is the entry). Before submitting: add your team name, college and member names; replace the 266 ms with your own measurement; export slide 2 as a PDF.

How to explain each part of the slide

- **The real problem**: queues, not bandwidth; port rules fail on encrypted traffic; speed changes.
- **Our idea**: find the cause first, act with safe fast rules, write a receipt for every change.
- **Bounded by design**: no content reading; intents are expiring leases with real guarantees; never promotes an uncertain flow; SAFE fallback.
- **Strip**: the six steps of the loop and the three speeds.
- **What makes it different / How we will prove it**: your four strengths and your test plan.

Round 2: the 10-slide deck

1. Title + one-line pitch
2. The physics of bufferbloat + your Day-0 numbers
3. Architecture
4. Classifier: features, abstention, accuracy vs port rules
5. Capacity estimator + fault localiser: the S2 and S6 plots
6. Receipts + safety state machine
7. Intent leases: API, budget, guarantees
8. Evidence: 5 arms × 9 scenarios with confidence intervals; freezes and stalls

9. Industry path: RDK-B/OpenWrt, fleet management, research tracks
10. Limitations + roadmap + demo QR code

18. Questions judges will ask, and how to answer

Q: CAKE already fixes bufferbloat. What did you add?

A: Fault diagnosis before acting, guaranteed-rate leases, payload-free classification and receipts. Then show the ablation chart against tuned SQM-CAKE and the real cake-autorate, including where we did not win.

Q: How do you tell a speed drop from a slow far-away server?

A: Near and far probes. Both slow with our queue empty means a WAN drop: cut. Only far slow means remote: hold. One probe disagreeing means a bad measurement: ignore it.

Q: What if the classifier is wrong?

A: We act only when it gives a single confident answer; otherwise normal treatment, never priority. We report coverage, selective accuracy and abstention separately. Users can correct it.

Q: Can a 15-second before/after prove a change helped?

A: No, and we don't claim that. Receipts can say 'inconclusive'. Proof comes from repeated randomised lab experiments.

Q: How do you guarantee no starvation?

A: HTB guaranteed rates within a budget during intents, and we measure every class's real progress.

Q: Do you read user data?

A: No. Only packet headers and timing. An optional, disclosed mode uses the router's own DNS/DHCP records.

Q: What happens if VeriQ crashes?

A: A watchdog switches to a conservative SAFE shaper. We measured the time it takes (scenario S8).

Q: How much CPU does it use?

A: Give your measured numbers on the Raspberry Pi 5 (and ideally a cheaper router chip).

Q: How would this scale to millions of homes?

A: The control loop is local. Fleet management would use the TR-369 (USP) standard, with staged rollouts. This part is designed, not built.

Q: What is emulated?

A: The ISP line in the lab (tbf/netem). It is labelled everywhere and checked on the physical kit.

Q: What about Wi-Fi?

A: We diagnose Wi-Fi problems when statistics are available; controlling Wi-Fi airtime is future work.

Q: What is the biggest limitation?

A: We infer the ISP queue but can't observe it, and shaping costs a few percent of speed. Say it calmly; it shows maturity.

19. Honest guarantees, limits, glossary and sources

What VeriQ promises (and tests)

1. Never inspects the content of forwarded traffic.
2. Never gives priority to an uncertain flow.
3. Intent leases always expire.
4. Guaranteed rates come from HTB, and progress is measured.
5. Every applied setting is versioned and checked.
6. A watchdog engages a SAFE shaper on failure.

Known limits (say these before the judges do)

- Shaping below the line speed costs a few percent of bandwidth.
- Download control depends on senders slowing down; aggressive UDP traffic can only be partly controlled.
- Classification without content is a probability, not a certainty.
- Wi-Fi control needs driver support.
- CAKE uses one CPU core; very fast (multi-gigabit) lines need more research.
- The ISP's queue is inferred, never directly seen.
- Emulated lines are not the same as real cable/fibre/5G networks.
- VeriQ must prove through ablations that it adds value beyond tuned CAKE and cake-autorate.

Glossary

Term	Meaning
Ablation	Removing one part to see how much it mattered
AQM	Active Queue Management: dropping or marking packets early to keep queues short
Bandwidth	How much data per second the line can carry
Baseline	The simple or existing method you compare against
Bufferbloat	Huge delay caused by oversized, full queues
CAKE	Advanced Linux qdisc: shaper + AQM + fairness + priority tins
Capacity	The real current speed of the internet line
Change-point detection	Statistics that notice a sudden change, like a speed drop
Confidence interval	A range that likely contains the true value
Conformal prediction	Gives a set of answers with a known chance of containing the truth
DASH	The method used by streaming video players to download chunks
DSCP	A priority label inside each packet

Term	Meaning
eBPF	Small, safe programs running inside the Linux kernel
Emulation	Imitating real equipment with software; must be labelled
fq_codel	Linux qdisc with per-flow lanes and early drops
Grafana	Dashboard software for charts
HTB	Linux qdisc with a tree of guaranteed and maximum rates
Intent / lease	A user's time-limited wish, like 'protect my call for 45 min'
IPv4 / IPv6	Old and new internet address formats
ISP	Internet Service Provider (Airtel, Jio, etc.)
Jitter	Variation in delay
Latency	Delay for a packet to travel
MOS	Mean Opinion Score: estimated call quality from 1 to 5
MVP	Minimum viable product: the core that must work
Namespace	An isolated network 'mini-computer' inside Linux
netem / tbf	Linux tools to fake delay and speed limits
p50 / p95 / p99	Median / 95th / 99th percentile
Packet	A small piece of data sent across the network
qdisc	Queueing discipline: the rules for a queue in Linux
QoS	Quality of Service: managing traffic so important things work well
QUIC / TLS	Modern encrypted transport protocols
RDK-B	Open-source software for broadband routers (Tata Elxsi works on it)
Receipt	VeriQ's short report on whether a change helped
RPM	Round-trips per minute under load: a responsiveness score
RTT	Round-trip time
Shaping	Deliberately sending slightly slower than the line to control the queue
SQM	Smart Queue Management: OpenWrt's CAKE/fq_codel setup
Starvation	A flow getting no useful bandwidth at all
tc	Linux traffic control command
TCP / UDP	Reliable transport (downloads, web) / fast transport (calls, games)
TR-181 / TR-369 (USP)	Broadband Forum standards for managing routers remotely
WAN / LAN	Internet side / home side
WebRTC	Technology used by browser video calls (Meet, Jitsi)

Reading list (about 10 hours)

1. Bufferbloat basics: bufferbloat.net 'What can I do about Bufferbloat'
2. 'Piece of CAKE' paper (Høiland-Jørgensen, Täht, Morton, 2018) and 'man tc-cake'

3. RFC 8289 (CoDel), RFC 8290 (FQ-CoDel)
4. cake-autorate README: github.com/lynxthecat/cake-autorate
5. 'man tc-netem', 'man tc-tbf', 'man tc-htb'
6. ITU-T G.107 (E-model) overview
7. IETF draft 'Responsiveness under Working Conditions' (draft-09)
8. Adams & MacKay 2007, 'Bayesian Online Changepoint Detection'
9. Angelopoulos & Bates, 'A Gentle Introduction to Conformal Prediction'

Main sources

- Tata Elxsi Q1 FY27 press release and investor presentation (14 July 2026); FY26 investor presentation
- Teliport S4 listings (GradPartners, Unstop) and a university notice; Season 2 report (6,000+ teams)
- Tata Elxsi RDK-B on Qualcomm 5G FWA (2024); NEURON platform page
- tc-cake manual; cake-autorate; OpenWrt qosify; IETF responsiveness draft-09; Comcast L4S deployment draft (2026)
- CESNET-QUIC22 dataset paper; WebRTC statistics (MDN); Cilium Bandwidth Manager documentation
- Full links are in docs/01, docs/05 and docs/06 of the project repository.

20. A rival team's prototype, and what we learned

On 12 October we studied a public GitHub repository (adaptive-qos-engine). It contains the exact Case 3 problem statement and was updated on 9 October 2026, so it is almost certainly **another team in your case**. It is a real, working prototype: about 5,900 lines of Python, 97 tests, scenario scripts and an evidence database.

> **Two rules**
>
> **Copy no code.** The repo has no licence (all rights reserved), and copying a competitor would be plagiarism and could get you disqualified. We only adopt general practices and published academic methods.
>
> **The competition is real.** Start building now, not after the shortlist.

Where VeriQ is stronger

Topic	Their prototype	VeriQ
Classifier	Per-packet length, TTL and timing; trained on 3 iperf3 test generators (99.1% on easy synthetic data)	Whole-flow features on real WebRTC and DASH traffic; says 'not sure'; tested on new traffic
Checking experience	3 pings, pass if under 60 ms	Real call freezes and video stalls; can say 'inconclusive'
Rollback	Fixed threshold, can undo a good change after the line itself got worse	Recalculated for current conditions
Cause diagnosis	None	WAN vs remote vs Wi-Fi vs bad probe
Guaranteed minimums	CAKE tuning trick, no reservation	HTB guaranteed rates + budget + measured progress
Priority labels	Rewrites DSCP (labels leak to the ISP)	Internal marks only
Intents	Natural-language parser with low confidence (the case warns against chat-only AI)	Structured, authenticated, expiring leases
Baselines	Normal router + a weak passive estimator	Real tuned SQM-CAKE and cake-autorate + ablations
Headline numbers	0.62 ms RTT (no internet delay emulated); 0.048 s adaptation timed from the estimator call	Realistic base delay; time measured from the actual drop; 10+ runs with confidence intervals

Good ideas we adopted (re-built our own way)

1. **Active probing on an idle line** (SLoPS, a published 2002 method), with fallback when no reflector server exists
2. **Estimator test list E1–E8**: static 100, static 20, drop, recovery, bursty traffic, many flows, plus our own remote-delay and lying-reflector tests
3. **Evidence ledger**: every run saves raw data plus settings; reports come only from it

4. **Claims check**: a script recomputes every number on your slides, and fails if any can't be reproduced
5. **Traceability matrix**: each case requirement → module → test → evidence file
6. **Fault-injection script and a discrepancy log** (write down every surprise)
7. **Deployment hygiene**: systemd service, install / verify / reset scripts, auto-detect interfaces
8. **Secure API**: token check, local-only access, rate limits, audit log
9. **Pitfall**: tc only shapes outgoing traffic, so always check that test traffic passes the shaped interface

> **Pitch sentence (never name the other team)**
>
> “Many CAKE-based controllers report ping. VeriQ reports what users feel, diagnoses the cause before acting, and compares itself honestly with the best open-source tools.”

Appendix: the other cases, and what the external review changed

Case 4 (LabelTwin), our earlier recommendation

For medical-device labeling we designed **LabelTwin**: every label becomes a 'digital twin' whose every element knows its data source, the regulation that requires it and the risk it controls. Regulations become tested rules, and AI drafts text and checks artwork but can only add findings, never approve. The full design is in docs/03 and docs/04 in case you ever switch.

What the external review improved in VeriQ (v1 → v2)

Version 1 said	Version 2 says
A 15-second canary 'proves' a change helped	Receipts can be inconclusive; proof only from controlled experiments
CAKE tins give minimum floors	CAKE gives soft preference; HTB gives real guarantees
95% conformal coverage	Coverage, selective accuracy and abstention reported separately
MOS as a measurement	MOS is estimated; freezes and stalls lead
'Never reads payload' + DNS hints	Precise wording; opt-in, disclosed tier
'cake-autorate-style' baseline	The real tuned cake-autorate + ablations
No cause diagnosis	Fault localiser before any action
Everything in scope	MVP spine vs ranked stretch goals

> **Final word**
>
> You now have a strong, honest design. What turns it into a winning entry is **evidence**: run the lab, build the MVP spine, measure carefully and present honestly. Good luck!